

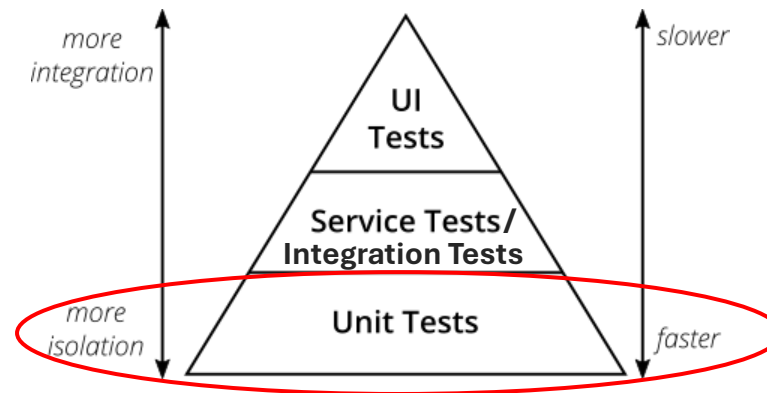
Testdoubles



Unit Test: Recap

Ein **Unit-Test** ist ein Test, der sich auf eine **isolierte Klasse** bezieht, die oft als *System Under Test* (SUT) bezeichnet wird.

Der Zweck von Unit-Tests ist es, zu überprüfen, ob der Code in einem SUT funktioniert.

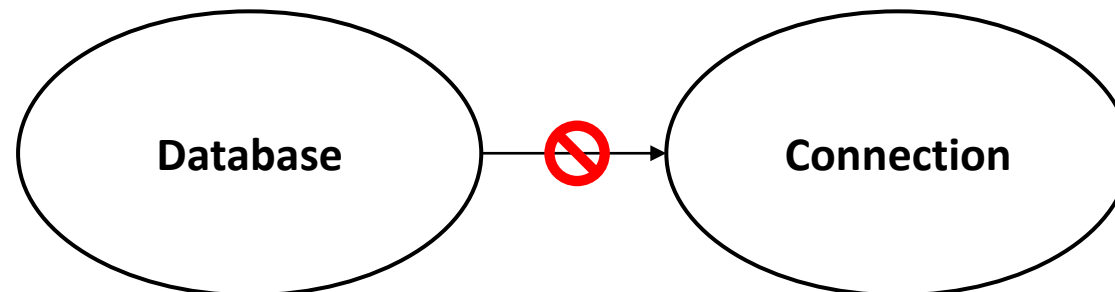


Isolierte Unit Tests

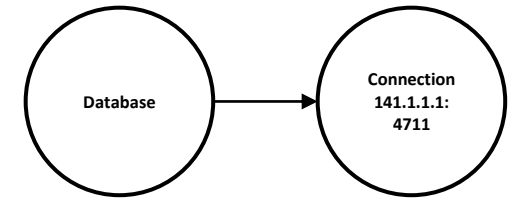
Beispiel:

Es gibt eine Klasse **Database**, die eine Abhängigkeit zur Klasse **Connection** hat ("Database verwendet Connection").

Im **Unit Test** soll die Klasse **Database** **isoliert getestet** werden. Um das zu erreichen, muss während des Unit Tests die Klasse **Connection** von der Klasse **Database** **entkoppelt** werden.



Beispiel: Hardcodiertes Objekt



```
class Database {
```

```
    private final Connection connection = new Connection("141.1.1.1", 4711); // hardcodierte Zuweisung  
                                         // nicht zur Laufzeit änderbar.
```

```
    public Person findPersonById(int personId) {  
        return connection.query(personId);  
    }  
}
```

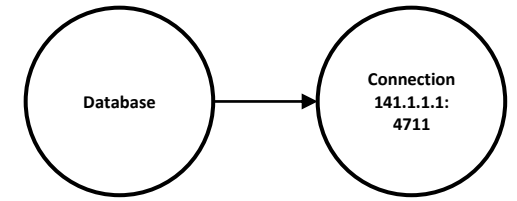
Problem:

Das Connection-Attribut erhält ein hardcodiertes Objekt zugewiesen, das sich während eines Unit Tests nicht ändern lässt.

Wenn wir einen Unit Test für die Klasse Database schreiben wollen, wollen wir nicht die **reale Datenbank** des Produktsystems verwenden sondern ein **Testdouble**!



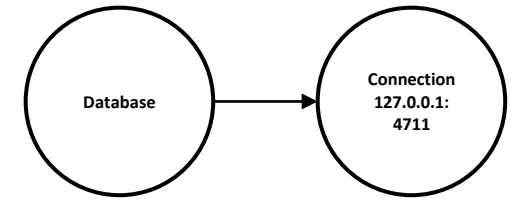
Dependency Injection 1/2



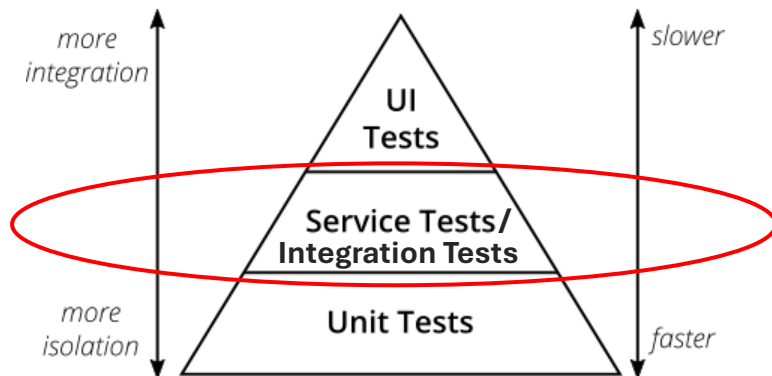
```
class Database {  
    private final Connection connection;  
  
    public Database(Connection connection) {  
        this.connection = connection; // connection wird nicht länger hardcodiert zugewiesen sondern dynamisch (aka Dependency Injection).  
    }  
  
    public Person findPersonById(int personId) {  
        return connection.query(personId); // Database verwendet die connection (Database ist abhängig von Connection).  
    }  
  
    // App für Produktion initialisieren und starten:  
    void main() {  
        String host = "141.1.1.1"; // Verwende IP-Adresse der Produktions-Datenbank.  
        int port = 4711; // Verwende Port der Produktions-Datenbank.  
        Connection connection = new Connection(host, port); // Erzeuge Dependency für Produktions-Database.  
        Database database = new Database(connection); // Dependency Injection.  
        database.findPersonById(1); // Database verwendet intern die Produktionsdatenbank für Abfragen.  
    }  
}
```

Dependency Injection (DI) bedeutet, dass eine **Abhängigkeit (ein Objekt)** über einen **Konstruktor oder Methode (Setter)** übergeben wird. Dadurch lässt sich das **Objekt austauschen**.

Dependency Injection 2/2 (Integrationstest)



```
class Database {  
    private final Connection connection;  
  
    public Database(Connection connection) {  
        this.connection = connection;  
    }  
  
    public Person findPersonById(int personId) {  
        return connection.query(personId);  
    }  
}
```



Integrationstest, der eine reale Testdatenbank verwendet:

```
class DatabaseTest {  
  
    private static Database database;  
  
    @BeforeAll  
    static void beforeAll() {  
        String host = "127.0.0.1"; // Verwende Adresse einer Testdatenbank.  
        int port = 4711; // Verwende Port einer Testdatenbank.  
  
        Connection testDatabaseConnection = new Connection(host, port);  
        database = new Database(testDatabaseConnection); // DI  
    }  
  
    @Test  
    public void findPersonByIdReturnsPersonWithSameId() {  
        int expectedPersonId = 42;  
        var person = database.findPersonById(expectedPersonId);  
        assertEquals(expectedPersonId, person.getId());  
    }  
}
```

Testdouble

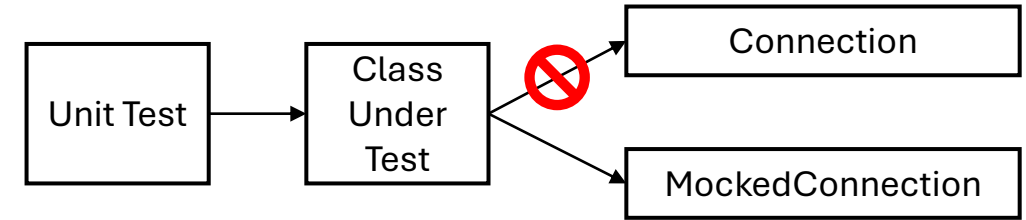


Ein **Testobjekt** (SUT) spricht häufig mit **anderen Objekten**, die als **Kollaborateure** oder **Abhängigkeiten** bezeichnet werden.

Um diese in einem **Unit-Test** zu **isolieren** und die **Kontrolle** über alle Aspekte des Tests zu behalten, ist es nützlich, die echten kollaborierenden Objekte durch "gefälschte" Ersatzobjekte, aka **Testdoubles**, zu ersetzen.

Testdoubles sehen aus **wie Objekte** der **originalen Klasse**, haben aber **keine Abhängigkeiten** zu anderen Objekten und **imitieren lediglich das Verhalten** des Originals.

Mocks



Mocks sind Objekte bzw. Testdoubles, die das Verhalten anderer Objekte nachahmen.

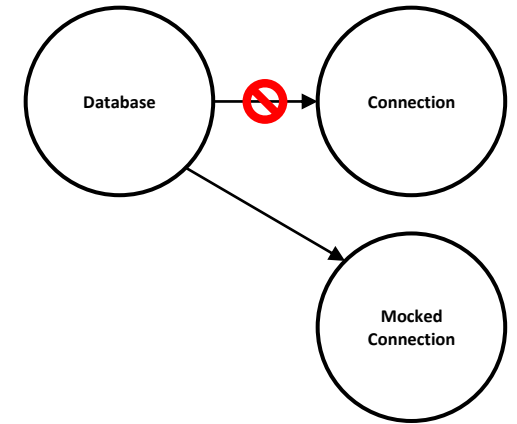
Mocks sind sinnvoll, um

- Tests unabhängig von anderen Teilen des Systems zu machen.
(z. B. externe Datenbanken)
- die Ausführungszeit von Tests zu erhöhen.
(z. B. Vermeiden vieler Anfragen übers Netzwerk)
- Fehler zu simulieren, die sich nur schwer reproduzieren lassen.
(z. B. Netzwerkfehler)

Beispiel: Mock 1/2

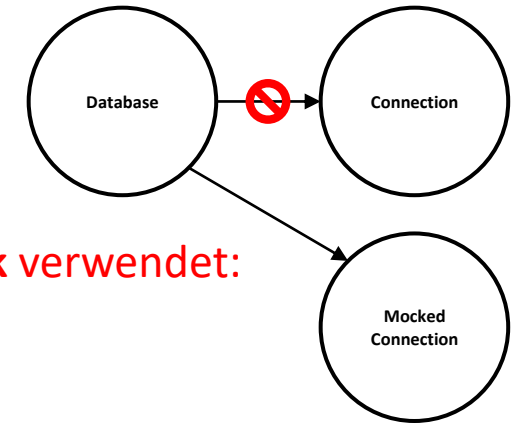
```
class Database {  
    private final Connection connection;  
  
    public Database(Connection connection) {  
        this.connection = connection;  
    }  
  
    public Person findPerson(int personId) {  
        return connection.query(personId);  
    }  
}
```

```
class DatabaseTest {  
    private static Database database;  
  
    @BeforeAll  
    static void beforeAll() {  
        database = new Database(new MockedDatabaseConnection());  
    }  
  
    @Test  
    public void findPersonByIdReturnsPersonWithSameId() {  
        int expectedPersonId = 42;  
        var person = database.findPerson(expectedPersonId);  
        assertEquals(expectedPersonId, person.getId());  
    }  
}
```

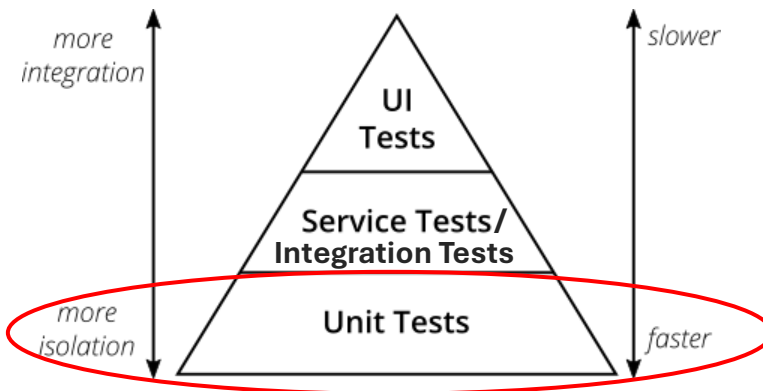


Durch Dependency Injection lassen sich Abhängigkeiten zu anderen Objekten durch Mocks ersetzen und ermöglichen so **unabhängige, isolierte Unit Tests!**

Beispiel: Mock 2/2 (Unit Test)



```
class Database {  
    private final Connection connection;  
  
    public Database(Connection connection) {  
        this.connection = connection;  
    }  
  
    public Person findPerson(int personId) {  
        return connection.query(personId);  
    }  
}
```



Unit Test, der keine reale Datenbank verwendet:

```
class DatabaseTest {  
    private static Database database;  
  
    @BeforeAll  
    static void beforeAll() {  
        database = new Database(new MockedDatabaseConnection());  
    }  
  
    @Test  
    public void findPersonByIdReturnsPersonWithSameId() {  
        int expectedPersonId = 42;  
        var person = database.findPerson(expectedPersonId);  
        assertEquals(expectedPersonId, person.getId());  
    }  
}
```

```
public class MockedDatabaseConnection implements Connection {  
    public Person query(int personId) {  
        return new Person(personId); // Dummy-Objekt für Testzwecke.  
    }  
}
```

Testdouble Arten



- **Dummy:** ein leeres Objekt, das in einem Aufruf übergeben wird (in der Regel nur, um einen Compiler zu befriedigen, wenn ein Methodenargument erforderlich ist).
- **Fake:** ein Objekt mit einer funktionalen Implementierung, aber normalerweise in einer vereinfachten Form, nur um den Test zu erfüllen (z. B. eine In-Memory-Datenbank).
- **Stub:** ein Objekt mit fest einkodiertem Verhalten, das für einen bestimmten Test (oder eine Gruppe von Tests) geeignet ist.
- **Mock:** ein Objekt mit der Fähigkeit, a) ein programmiertes erwartetes Verhalten zu haben und b) während seiner Lebensdauer auftretende Interaktionen zu verifizieren.
- **Spy:** ein Mock, das als Stellvertreter für ein bestehendes reales Objekt erstellt wird; einige Methoden können stubbed sein, während nicht-stubbed Methoden an das reale Objekt weitergeleitet werden.



Mockito

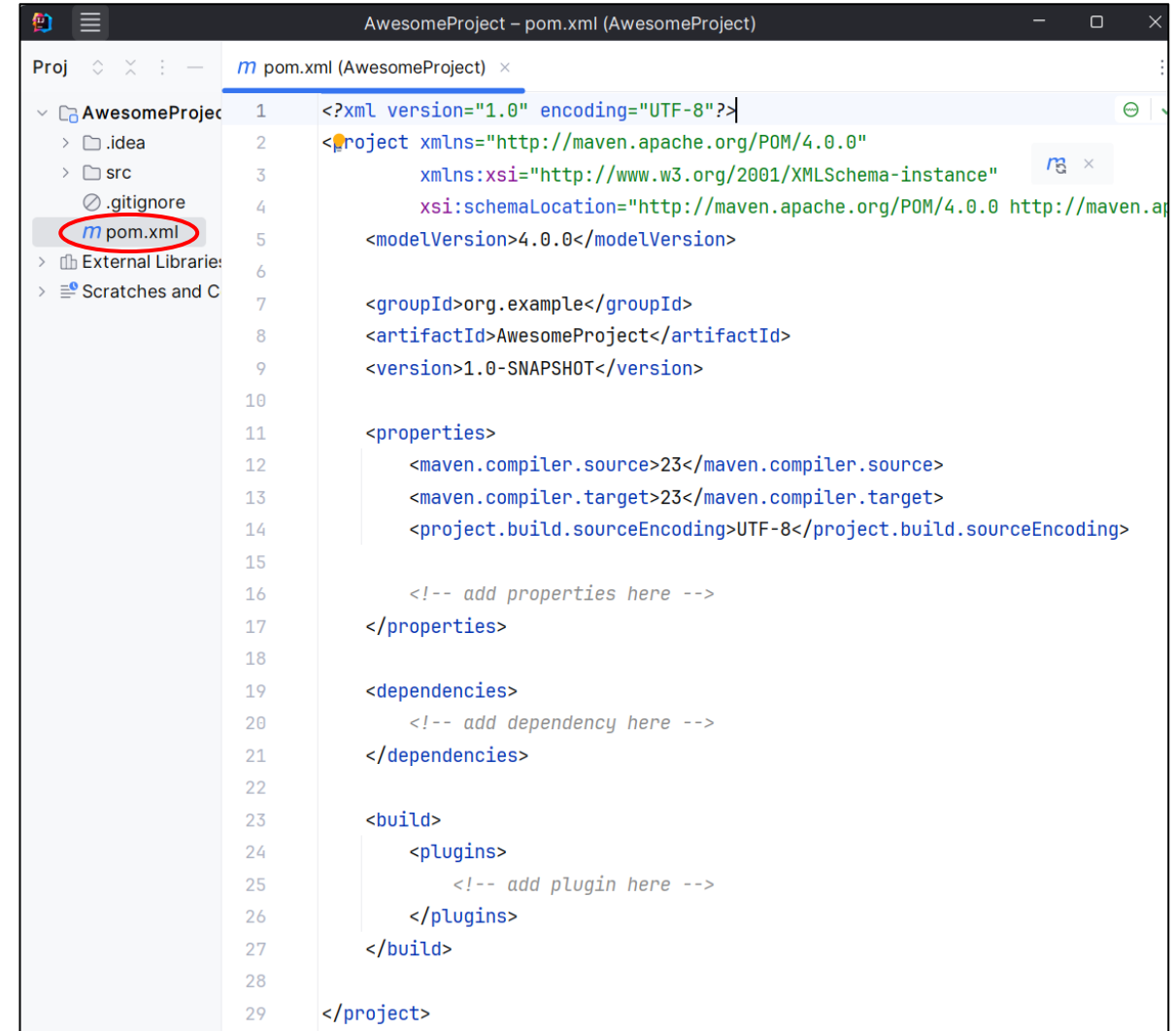
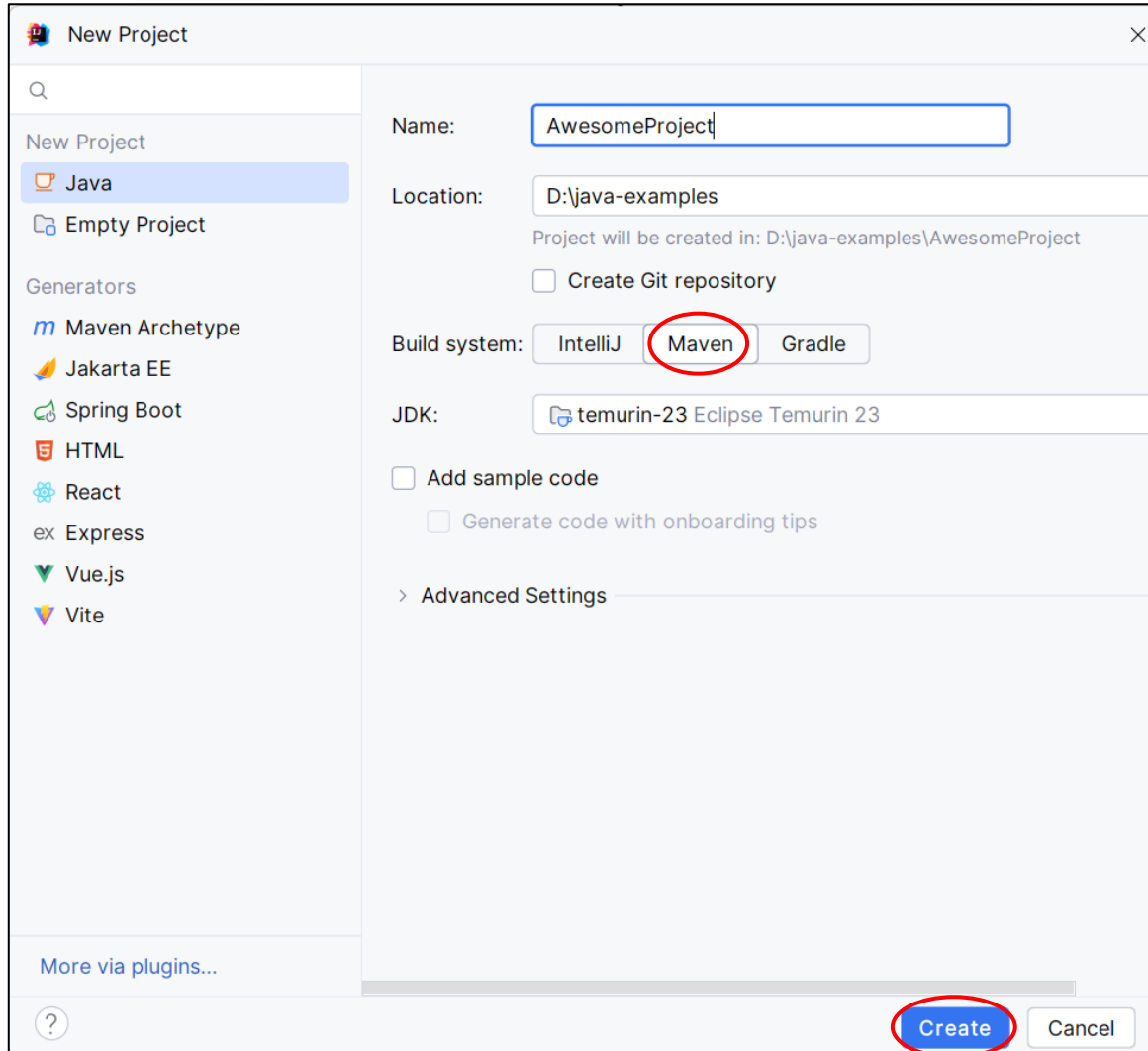


Mockito ist ein Mocking-Framework für Java, um unter anderen Testdoubles zu erstellen.

"Mit Mockito bekommt man keinen Kater, weil die Tests gut lesbar sind und Mockito saubere Fehlermeldungen produziert."

Quelle: <https://site.mockito.org/>

Mockito mit Maven 1/2



Mockito mit Maven 2/2

```
<properties>
  <!-- Java settings -->
  <maven.compiler.source>25</maven.compiler.source>
  <maven.compiler.target>25</maven.compiler.target>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>

  <!-- ... other properties, e.g., for JUnit -->

  <!-- Shared version properties -->
  <mockito.version>5.17.0</mockito.version>
  <maven-surefire-plugin-version>3.5.3</maven-surefire-plugin-version>
</properties>

<dependencies>
  <!-- ... other dependencies, e.g., JUnit -->

  <!-- Mockito -->
  <dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-core</artifactId>
    <version>${mockito.version}</version>
    <scope>test</test>
  </dependency>
  <!-- Needed to remove the Mockito warnings. -->
  <dependency>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-surefire-plugin</artifactId>
    <version>${maven-surefire-plugin-version}</version>
    <scope>test</test>
  </dependency>
</dependencies>
```

```
<build>
  <plugins>
    <!-- Removes inline-mock warning when using Mockito -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>${maven-surefire-plugin-version}</version>
      <configuration>
        <!-- Dynamically load mockito as an agent. -->
        <!-- A Java agent is basically a piece of software that can modify the behavior of the
        Java Virtual Machine (JVM) during runtime. In contrast to normal libraries or dependencies that
        are called directly by the application, agents "hack" into the JVM and can therefore make more
        far-reaching changes (e.g., byte code manipulations). -->
        <argLine>
          -XX:+EnableDynamicAgentLoading
          -Dnet.bytebuddy.experimental=true
          -javaagent:${user.home}/.m2/repository/org/mockito/mockito-core/${mockito.version}/mockito-core-${mockito.version}.jar
        </argLine>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Mockito: verify

Ein Mockito-Objekt **zeichnet** alle **Interaktionen auf**, d. h. alle Methodenaufrufe.

Mittels **verify** lässt sich im Unit Test prüfen, ob bestimmte **Methoden inklusive Parameter** auf dem Mock **aufgerufen wurden**.

verify prüft **ähnlich wie assert-**Methoden, ob eine Bedingung erfüllt ist, und zwar ob eine Methode aufgerufen wurde. Wenn nicht, dann schlägt der Testfall fehl.

```
import org.junit.jupiter.api.Test;
```

```
import java.util.List;
```

```
// Importiere Mockito statisch, sodass der Code klarer ist.
```

```
import static org.mockito.Mockito.mock;
```

```
import static org.mockito.Mockito.verify;
```

```
class VerifyTest {
```

```
    @Test
```

```
    void demo() {
```

```
// FYI: List würde man normalerweise nicht mocken (nur Demo).
```

```
List<String> mockedList = mock();
```

```
// Interagiere mit dem Mock-Objekt.
```

```
mockedList.add("one");
```

```
mockedList.clear();
```

```
// Verifiziere Interaktion mit Mock-Objekt (quasi wie assert).
```

```
verify(mockedList).add("one"); // wurde add mit "one" aufgerufen?
```

```
verify(mockedList).clear(); // wurde clear() aufgerufen?
```

```
    }
```

```
}
```

Mockito: verify – times()

Per Default prüft verify, ob eine Methode **genau einmal** aufgerufen wurde.

```
class ListTest {  
  
    @Test  
    void multipleCalls() {  
        List<String> mockList = mock();  
  
        mockList.add("Sara");  
        mockList.add("Sara");  
  
        verify(mockList).add("Sara");  
    }  
}
```



org.mockito.exceptions.verificatioon.TooManyActualInvocations:
list.add("Sara");
Wanted 1 time:
But was 2 times:

```
class ListTest {  
  
    @Test  
    void multipleCalls() {  
        List<String> mockList = mock();  
  
        mockList.add("Sara");  
        mockList.add("Sara");  
  
        verify(mockList, times(2)).add("Sara");  
    }  
}
```



Mockito: when (Stubbing)

```
class StubbingTest {
```

```
    @Test
```

```
    void demo() {
```

```
        // Man kann auch konkrete Klassen mocken, nicht nur Interfaces.
```

```
        // FYI: LinkedList würde man normalerweise nicht mocken (nur Demo).
```

```
        LinkedList<String> mockedList = mock();
```

```
        // Stubbing
```

```
        when(mockedList.get(0)).thenReturn("first");
```

```
        when(mockedList.get(1)).throw(new RuntimeException());
```

```
        // Ausgabe: "first".
```

```
        IO.println(mockedList.get(0));
```

```
        // Wirft RuntimeException.
```

```
        IO.println(mockedList.get(1));
```

```
        // Ausgabe: "null", da .get(999) nicht gestubbed wurde (gibt stattdessen einen typabhängigen Default-Wert zurück).
```

```
        IO.println(mockedList.get(999));
```

```
        // Obwohl es möglich ist, einen Stubbed-Aufruf zu verifizieren, ist es überflüssig.
```

```
        verify(mockedList).get(0);
```

```
        // FYI: Stubbe nur Methoden, die auch von der zu testenden Klasse aufgerufen bzw. verwendet werden.
```

```
    }
```

```
}
```

Mit **when** lässt sich das Verhalten eines Mocks programmieren.

Angenehm: Mockito stellt für alle Methoden, die nicht mit **when** überschrieben werden, eine Standard-Implementierung bereit, so dass man nur die Methoden in einer Klasse oder in einem Interface mocken muss, die man gerade benötigt bzw. testen möchte.



Mockito: Argument Matchers

```
class ArgumentMatchers {  
  
    @Test  
    void demo() {  
        List<String> mockedList = mock();  
  
        // Stubbing mit anyInt()-Matcher  
        when(mockedList.get(anyInt())).thenReturn("element"); // wenn get mit irgendeinen int-Wert aufgerufen, dann gebe "element" zurück.  
  
        // Stubbing ist nur notwendig, wenn der Rückgabewert eine Rolle spielt (siehe .thenReturn).  
        // Ansonsten lassen sich die Methoden der gemockten Klasse auch einfach direkt aufrufen:  
        mockedList.add("Sara"); // Aufruf ohne stubbing  
  
        // Ausgabe: "element"  
        IO.println(mockedList.get(999));  
  
        // Ausgabe: false  
        IO.println(mockedList.contains("Sara"));  
  
        // Argument Matchers funktionieren auch mit verify.  
        verify(mockedList).get(anyInt());  
  
        verify(mockedList).add(anyString());  
    }  
}
```

Argument Matcher erlauben es, dass ein Mock auf irgendwelche Parameter reagiert. D. h., der übergebene Wert spielt keine Rolle, Hauptsache der Typ stimmt überein.

Mockito: Argument Matchers – Reihenfolge

Die Reihenfolge bei "when mit Argument Matcher" ist relevant.

```
class ListTest {  
  
    @Test  
    void multipleWhenWithMatcher() {  
        List<String> mockList = mock();  
  
        when(mockList.get(1)).thenReturn("Velvet");  
        // Folgender Matcher überschreibt vorherige Zeile.  
        when(mockList.get(anyInt())).thenReturn("Sara");  
  
        assertEquals("Velvet", mockList.get(1));  
        assertEquals("Sara", mockList.get(2));  
    }  
}
```



org.opentest4j.AssertionFailedError:
Expected :Velvet
Actual :Sara

```
class ListTest {  
  
    @Test  
    void multipleWhenWithMatcher() {  
        List<String> mockList = mock();  
  
        when(mockList.get(anyInt())).thenReturn("Sara");  
        when(mockList.get(1)).thenReturn("Velvet");  
  
        assertEquals("Velvet", mockList.get(1));  
        assertEquals("Sara", mockList.get(2));  
    }  
}
```



Mockito: Argument Matchers all or nothing

Achtung:

Sobald man einen Argument Matcher verwendet, müssen **alle** Parameter der Methode einen Argument Matcher verwenden!

*// **Falsch:** es wird eine Exception ausgelöst, weil das dritte Argument ohne einen Argument-Matcher angegeben wird.*

❌ `verify(mock).someMethod(anyInt(), anyString(), "third argument"); // falsch`

*// **Korrekt:** `eq()` ist auch ein Argument-Matcher, der für "equals" steht.*

✅ `verify(mock).someMethod(anyInt(), anyString(), eq("third argument")); // korrekt`

Mockito: Argument Matchers Kontext

Achtung:

Verwende Argument Matcher nur im Zusammenhang mit **when** oder **verify**!

```
Service peopleServiceMock = mock();
```

✅ *when*(peopleServiceMock.find(*any*())); // *korrekt, da when.*

```
peopleServiceMock.save(new Person("Sara"));
```

❌ peopleServiceMock.save(*any*()); // *falsch, da kein when oder verify.*

✅ *verify*(peopleServiceMock).save(*any*()); // *korrekt, da verify.*

// FYI: Folgende Schreibweise wäre notwendig, falls es *mehrere Überladungen* für die Methode save gäbe:

✅ *verify*(peopleServiceMock).save(*any*(Person.class)); // *korrekt, da verify.*

staticMock

Statische Methoden sind in der objektorientierten Welt problematisch, da sich diese in Unit Tests **nur schwer austauschen bzw. testen** lassen.

Beispiel:

Die statische Java-Methode **LocalDate.now()** gibt z. B. das aktuelle Datum zurück.



Will man in einem Unit Test testen, wie sich eine Applikation an Weihnachten verhält, muss man irgendwie dafür sorgen, dass `LocalDate.now()` immer das Datum von Weihnachten zurückgibt...



staticMock Beispiel



```
import java.time.LocalDate;

final class Christmas {

    static String getMessageOfDay() {
        LocalDate today = LocalDate.now(); // statische Methode 🗓️
        boolean isChristmas = today.getMonthValue() == 12 &&
                               today.getDayOfMonth() == 25;

        if (isChristmas) {
            return "Ho Ho Ho!";
        } else {
            return "Christmas is coming...";
        }
    }
}
```

FYI: Der Parameter `InvocationOnMock::callRealMethod` in `mockStatic` gibt an, dass die echten Methodenaufrufe verwendet werden sollen, wenn eine Methode auf dem Mock-Objekt aufgerufen wird, es sei denn, es gibt eine explizite Anweisung (z. B. `when(...).thenReturn(...)`), die das Verhalten überschreibt.

Das bedeutet, dass für alle Methoden von `LocalDate` die echte Implementierung aufgerufen wird, außer eine Methode wurde mit `when(...)` überschrieben.

```
import org.junit.jupiter.api.Test;
import org.mockito.MockedStatic;
import org.mockito.invocation.InvocationOnMock;

import java.time.LocalDate;

import static com.google.common.truth.Truth.assertThat;
import static org.mockito.Mockito.mockStatic;

class ChristmasTest {

    @Test
    void isChristmas() {
        try (MockedStatic<LocalDate> staticMock = mockStatic(LocalDate.class, InvocationOnMock::callRealMethod)) {
            // Arrange
            LocalDate christmasDate = LocalDate.of(0, 12, 25); // LocalDate.now() soll dies im Unit Test zurückgeben.
            staticMock.when(LocalDate::now).thenReturn(christmasDate); // rewrite the static method now()

            // Act
            String message = Christmas.getMessageOfDay();

            // Assert
            assertThat(message).isEqualTo("Ho Ho Ho!");
        }

        // Alternative Schreibweise:
        @Test
        void isNotChristmas() {
            try (MockedStatic<LocalDate> _ = mockStatic(LocalDate.class, InvocationOnMock::callRealMethod)) {
                // Arrange
                LocalDate notChristmasDate = LocalDate.of(0, 7, 30); // LocalDate.now() soll dies im Unit Test zurückgeben.
                // Die Variable staticMock wird hier nicht verwendet, muss aber trotzdem in try-with-resources zugewiesen werden.
                // Für Variablen bzw. Parameter, die nicht verwendet werden, hat sich etabliert ein _ als Name zu verwenden.
                when(LocalDate.now()).thenReturn(notChristmasDate); // rewrite the static method now()

                // Act
                String message = Christmas.getMessageOfDay();

                // Assert
                assertThat(message).isEqualTo("Christmas is coming...");
            }
        }
    }
}
```

staticMock Fallstricke



Achtung: Das Stubbing wird von Mockito vor allen anderen Methodenaufrufen zuerst durchgeführt. Mockito setzt deshalb voraus, dass der gesamte Stub inklusive aller Parameter wie z. B. bei `when(...).thenReturn(...)` vollständig initialisiert wurde.

```
import java.time.LocalDate;

final class Christmas {

    static String getMessageOfTheDay() {
        LocalDate today = LocalDate.now(); // statische Methode 🎅
        boolean isChristmas = today.getMonthValue() == 12 &&
                               today.getDayOfMonth() == 25;

        if (isChristmas) {
            return "Ho Ho Ho!";
        } else {
            return "Christmas is coming...";
        }
    }
}
```

```
import org.junit.jupiter.api.Test;
import org.mockito.MockedStatic;
import org.mockito.invocation.InvocationOnMock;

import java.time.LocalDate;

import static com.google.common.truth.Truth.assertThat;
import static org.mockito.Mockito.mockStatic;

class ChristmasTest {

    @Test
    void isNotChristmas() {
        try (MockedStatic<LocalDate> staticMock = mockStatic(LocalDate.class, InvocationOnMock::callRealMethod)) {
            // Arrange
            // Mockito benötigt als Parameter für thenReturn ein fertiges Objekt.
            // Wenn stattdessen direkt LocalDate.of als Parameter übergeben wird, erhält man eine Exception:
            // "Unfinished stubbing detected here"
            staticMock.when(LocalDate::now).thenReturn(LocalDate.of(0, 7, 30)); // Schlägt fehl. 🚫

            // Korrekt:
            LocalDate notChristmasDate = LocalDate.of(0, 7, 30);
            staticMock.when(LocalDate::now).thenReturn(notChristmasDate); // Funktioniert! 👍

            // Act
            String message = Christmas.getMessageOfTheDay();

            // Assert
            assertThat(message).isEqualTo("Ho Ho Ho!");
        }
    }
}
```


staticMock vs Dependency Injection

Statische Methoden lassen sich manchmal durch geschicktes Dependency Injection ersetzen.

```
import java.time.LocalDate;

@FunctionalInterface
interface DateProvider {
    LocalDate getCurrentDate();
}

class Christmas {

    private DateProvider dateProvider;

    // Im Unit Test kann ein spezieller dateProvider übergeben (injiziert) werden.
    Christmas(DateProvider dateProvider) {
        this.dateProvider = dateProvider;
    }

    String getMessageOfDay() {
        LocalDate today = dateProvider.getCurrentDate(); // verwende injiziertes Objekt.
        boolean isChristmas = today.getMonthValue() == 12 && today.getDayOfMonth() == 25;
        return isChristmas ? "Ho Ho Ho!" : "Christmas is coming...";
    }
}
```

```
import org.junit.jupiter.api.Test;
import java.time.LocalDate;
import static com.google.common.truth.Truth.assertThat;

class ChristmasTest {

    @Test
    void isChristmas() {
        // Arrange
        LocalDate christmasDate = LocalDate.of(2025, 12, 25);
        Christmas christmas = new Christmas() -> christmasDate; // Dependency Injection 

        // Act
        String actual = christmas.getMessageOfDay();

        // Assert
        assertThat(actual).isEqualTo("Ho Ho Ho!");
    }

    @Test
    void isNotChristmas() {
        // Arrange
        LocalDate notChristmasDate = LocalDate.of(2025, 12, 1);
        Christmas christmas = new Christmas() -> notChristmasDate; // Dependency Injection 

        // Act
        String actual = christmas.getMessageOfDay();

        // Assert
        assertThat(actual).isEqualTo("Christmas is coming...");
    }
}
```

Mockito: Weitere Features

- Bedingungen für Parameter festlegen (argThat und Custom Argument Matchers).
- Prüfen, wie oft eine Methode aufgerufen wurde (times, never).
- Prüfen, ob eine Methode innerhalb einer bestimmten Zeit antwortet (Timeout).
- Aufrufe an ein reales Objekt spionieren (spy).

Und vieles mehr, siehe [Doku](#).

Bedenke: Mit großer Macht kommt große Verantwortung!





Best Practices: Mocks

- Mocke keine Typen (Klassen), die du nicht besitzt (z. B. von externen Bibliotheken).
- Mocke keine Value Objects (i. d. R. immutable Klassen wie z. B. [record](#)).
- Mocke nicht alles!
 - Übertriebenes Mocking führt zu komplizierten und schwer wartbaren Tests.
 - Vermeide Mocks, die andere Mocks erstellen und zurückgeben.
 - Mocking-Frameworks, wie [Mockito](#), sind nicht unbedingt nötig.

Alternative: Dependency Injection mit Interfaces und selbstgeschriebene Mock-Klassen.

Mockito: Wann verwenden?

Die Stärke von Mockito ist es, **komplexe Klassen oder Interfaces** mocken zu können, weil durch Mockito nicht für jeden Mock eine neue Klasse mit allen Methoden implementiert werden muss.

Mockito ist insbesondere für **Input/Output (I/O)**-Klassen nützlich, wie z. B. Netzwerkkommunikation, Datenbankverbindungen oder Dateiverarbeitung, um einen **isolierten Unit Test** zu schreiben, der frei von **potenziellen I/O-Fehlern** ist, wie z. B. Verbindungsverlust, Hardwarefehler und Timeouts.

Trotzdem sollten **zusätzlich** noch **Integrationstests** ohne Mocks geschrieben werden, um das Zusammenspiel mit reale I/O-Klassen zu testen, da Mocks eventuell von falschen oder unrealistischen Annahmen ausgehen und auch nicht die reale Hardware testen.

Mockito: Links

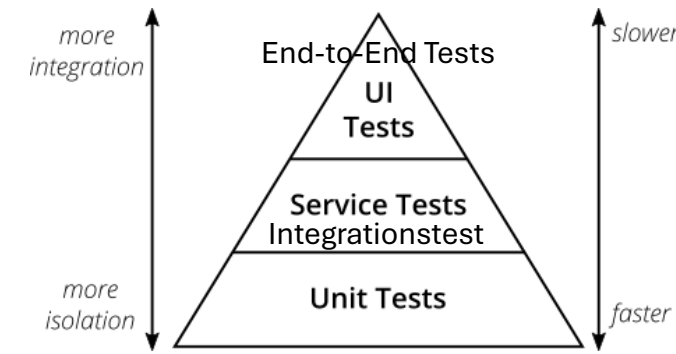
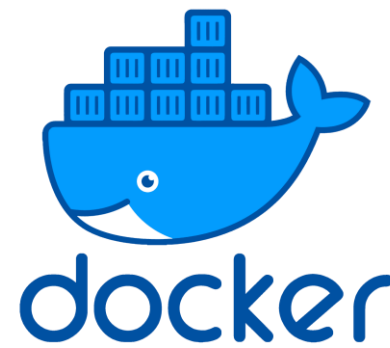
Webseite:

<https://site.mockito.org/>

Dokumentation:

<https://javadoc.io/doc/org.mockito/mockito-core/latest/org/mockito/Mockito.html>

Exkurs: Testcontainers



In der Softwareentwicklung werden inzwischen häufig Testcontainers eingesetzt. Diese basieren auf Docker-Container und helfen, **automatisierte Integrationstests** zu verwalten und durchzuführen.

Testcontainers starten **für Testzwecke echte Software-Umgebungen inklusive aller Abhängigkeiten**, wie Datenbanken, Message-Broker und weitere Services.

Das Ziel ist, **Integrationstests und End-to-End-Tests realistischer** und zuverlässiger zu gestalten, ohne dass externe Dienste manuell aufgesetzt oder gemockt werden müssen.

Ein großer Vorteil ist ihre **Portabilität**: da ein Testcontainer sämtliche Abhängigkeiten enthält, kann er leicht auf anderen Systemen erstellt oder übertragen werden (z. B. Entwicklerrechner oder Testsystem).